CS108 Project Report

Mini Game Hub

Bash · Python · Pygame

Lucky Gupta and Duduku Jaswanth Balaji

April 2025

Contents

1 Introduction **3**

2 Running Instructions **3**

3 Project Layout **3**
3.1 Authentication (`main.sh`) . . . 3
3.2 Game Engine (`game.py`) . . . 4
3.3 Game Menu . . . 4
3.4 In-Game UI . . . 4

4 Class Structure **5**
4.1 Base Class — `BoardGame` . . . 5

5 Games **5**
5.1 Tic-Tac-Toe (`games/tictactoe.py`) . . . 5
5.2 Connect Four (`games/connect4.py`) . . . 6
5.3 Othello (`games/othello.py`) . . . 6

6 Authentication System (`main.sh`) **6**

7 Leaderboard (`leaderboard.sh`) **7**

8 Modules Used **7**
8.1 pygame-ce . . . 7
8.2 NumPy . . . 7
8.3 Matplotlib . . . 7
8.4 pathlib, sys, os . . . 8
8.5 subprocess . . . 8
8.6 csv, datetime . . . 8

9 Directory Structure **8**

10 Basic and Advanced Features **8**

10.1 Basic Features 8
10.2 Advanced Features 9

11 Hurdles Faced **9**

12 What I Would Do Differently With 10 More Hours **10**

1 Introduction

Mini Game Hub is a Two-player game hub built as part of the CS108 course (Spring semester 2026). It integrates Bash scripting for user authentication and Python (Pygame) for graphical gameplay, enabling two authenticated players to select a game from an interactive menu, play via a Pygame GUI window, and have their results recorded on a persistent leaderboard.

The project focuses on modularity, code quality, and a clean user experience. It supports the following features:

- Two-player authentication with SHA-256 hashed passwords stored in `users.tsv`.
- An interactive Pygame game-selection menu with a single-click launch.
- Three fully playable board games rendered with Pygame: Tic-Tac-Toe (10×10), Othello (8×8), and Connect Four (7×7).
- Persistent game history recorded in `history.csv` after every game.
- A formatted terminal leaderboard rendered by a Bash script, sortable by wins, losses, or win/loss ratio.
- A post-game screen that prompts players to select a leaderboard sort metric before displaying statistics.

2 Running Instructions

- Ensure Python 3.10+ and `pip` are installed.
- Install the required Python libraries:

```
pip install pygame-ce numpy matplotlib
```

- Navigate to the project root directory and run:

```
bash main.sh
```

- Follow the authentication prompts for both players, then select a game from the Pygame menu that appears.

3 Project Layout

3.1 Authentication (`main.sh`)

`main.sh` is the entry point for the hub. It authenticates two players before launching the Python game engine.

- Prompts for two distinct usernames and passwords sequentially.
- Prevents both players from using the same username.

- Hashes passwords with SHA-256 using `sha256sum` before storing them in `users.tsv` — plaintext passwords are never stored.
- On login, hashes the typed password and compares it against the stored hash; re-prompts on mismatch.
- If a username does not exist, offers registration and updates `users.tsv` accordingly.
- Once both players are authenticated, calls:

```
python3 game.py <username1> <username2>
```

3.2 Game Engine (`game.py`)

`game.py` receives both authenticated usernames and manages the full Python-side flow.

- Initialises a 700×800 Pygame window.
- Displays the game-selection **MenuScreen**: a plain vertical list of games, single click to launch — no hover card effects, no confirmation button.
- Runs the selected game via `run_game()`, which manages the Pygame event loop generically across all three games.
- After each game, displays the **PostGameScreen** in two phases:
  1. Shows the result and three sort-metric buttons (Wins / Losses / W/L Ratio). On selection, writes a row to `history.csv` and calls `leaderboard.sh`.
  2. Prompts both players to play another game or quit.

3.3 Game Menu

The menu renders each game as a plain text row. A single click on any row launches that game immediately. On hover, a subtle background tint and a thin left accent bar appear. There are no card boxes, no play button, and no double-click requirement.

3.4 In-Game UI

Every game uses a shared 700×800 window with the following layout:

Region	Height
Top UI bar	80 px
Board area (centred, with 20 px margin)	680 px

The top bar (implemented in `base_game.py` as `draw_top_bar()`) shows:

- **Left**: game name (small, dimmed).
- **Centre**: P1name **vs** P2name in player colours.
- **Right**: whose turn it is (with a coloured dot), or the result and keyboard hints once the game ends.

4 Class Structure

4.1 Base Class — `BoardGame`

All games inherit from the `BoardGame` class defined in `game.py`. Each method in the class is overridable subclass.

Shared state attributes:

- `player_names` — dict mapping player integer (1 or 2) to username string.
- `current_player` — integer (1 or 2) indicating whose turn.
- `board` — a NumPy array representing the game board.
- `winner` — `1`, `2`, `0` (draw), or `None` (ongoing).
- `move_count` — total moves made.

Concrete helpers (available to all subclasses):

- `switch_turn()` — flips `current_player` between 1 and 2.
- `is_game_over()` — returns `True` if `winner` is not `None`.
- `current_player_name()` — returns the active player's username.
- `get_result_string()` — human-readable result for the post-game screen.
- `draw_top_bar(surf)` — renders the shared top UI bar.
- `get_font(size, bold)` — returns a `SysFont`.

Methods subclasses must override:

Method	Purpose
`reset()`	Initialise board to starting state, reset all state.
`check_winner()`	Return 1, 2, 0, or None using NumPy operations.
`draw_board(surf)`	Render full frame onto `surf`; must call `draw_top_bar()`.
`handle_click(pos)`	Convert pixel click `(x,y)` to a game move.

5 Games

5.1 Tic-Tac-Toe (`games/tictactoe.py`)

- **Board**: 10×10 NumPy integer array. Values: 0 = empty, 1 = Player 1 (X), 2 = Player 2 (O).
- **Win condition**: 5 marks in a row — horizontal, vertical, or diagonal.
- **Cell size**: `best_cell_size(10) = 660 // 10 = 66 px`.

5.2 Connect Four (`games/connect4.py`)

- **Board**: 7×7 NumPy array. Values: 0 = empty, 1 = P1, 2 = P2.
- **Win condition**: 4 coins in a row.
- **Gravity**: `np.where(board[:, col] == 0)[0][-1]` finds the lowest empty row — no Python loop.
- **Cell size**: `best_cell_size(7) = 660 // 7 = 94 px`.
- **Coins colours**: one fixed colour per player (red for P1, blue for P2); no cycling.
- **Drop preview**: a fully opaque ghost coin is rendered in the gap between the top bar and the board, clamped so it is always fully visible.

Win Detection

Uses the same sliding-consec approach as Tic-Tac-Toe but with `WIN_LEN = 4`. All four directions checked with `broadcasting` and `np.argwhere`.

5.3 Othello (`games/othello.py`)

- **Board**: 8×8 NumPy array. Values: 0 = empty, 1 = Black (P1), 2 = White (P2).
- **Starting position**: two Black and two White discs in the centre four cells.
- **Cell size**: `best_cell_size(8) = 660 // 8 = 82 px`.
- **Board appearance**: classic green felt background with corner star-point dots.

Turn Skipping

After each move, if the next player has no valid moves their turn is skipped back. If neither player can move, `check_winner()` counts discs with `np.sum(board == player)` and declares the result.

UI Features

- **Valid move hints**: small dots on all legal cells so players always know where they can place.
- **Ghost disc**: semi-transparent disc previews placement on hover.
- **Flip ring**: gold ring drawn around discs flipped on the last move.
- **Disc count bar**: proportional fill bar below the board showing each player's current count.

6 Authentication System (`main.sh`)

- Uses `declare` (not `eval`) for safe variable assignment — prevents code injection from special characters in usernames.
- Passwords are hashed with:

```
hashed=$(echo -n "$password" | sha256sum | cut -d' ' -f1)
```

- `grep -F` with a literal tab separator prevents regex false-matches on special-character usernames.
- Password confirmation on registration uses its own inner `while true` loop, so a mismatch re-prompts for the password rather than looping back to the username prompt.
- `touch "$USERS_FILE"` at startup creates the file if it does not exist, preventing `grep` errors on first run.

7 Leaderboard (`leaderboard.sh`)

`leaderboard.sh` is called by `game.py` after each game, with the sort metric chosen by the players as an argument (`wins`, `losses`, or `ratio`).

- Reads `history.csv`, groups results by game name.
- For each game, uses `awk` to accumulate wins and losses per player, computes win/loss ratio, and sorts the output using the passed metric.
- Prints a formatted bordered table to the terminal with columns: Rank, Player, Wins, Losses, W/L Ratio.
- Handles missing or empty history gracefully with a friendly message.

8 Modules Used

8.1 pygame-ce

Pygame Community Edition pygame is used for all graphical rendering across the game hub: the menu screen, all three game boards, the top UI bar, and the post-game screens. It handles the event loop, keyboard and mouse input, and font rendering.

8.2 NumPy

NumPy numpy is used for all game board representations and win detection. All boards are stored as NumPy integer arrays. Win conditions are checked using vectorized operations (`np.cumsum`, `np.where`, `np.maximum.accumulate`, `np.diagonal`, `np.fliplr`) — no Python loops over cells.

8.3 Matplotlib

Matplotlib matplotlib is listed as an allowed module. It is available for post-game analytics visualisation (e.g. bar chart of top players, pie chart of most-played games from `history.csv`).

8.4 pathlib, sys, os

Standard library modules used for path manipulation, argument parsing, and cross-platform file handling.

8.5 subprocess

Used in `game.py` to call `leaderboard.sh` with `subprocess.run(["bash", LEADERBOARD_SH, sort_by])`.

8.6 csv, datetime

Standard library modules used in `game.py` for writing rows to `history.csv` after each game.

9 Directory Structure

```
hub/
|---- main.sh              # Entry point: authentication + launch
|---- game.py              # Game engine: menu, loop, history,
   leaderboard
|---- leaderboard.sh       # Terminal leaderboard (Bash + awk)
|---- users.tsv            # Username <TAB> SHA-256 hash
|---- history.csv          # Winner, Loser, Date, Game
|---- games/
|    |---- tictactoe.py     # TicTacToe (10x10, 5 in a row)
|    |---- othello.py       # Othello (8x8, flip discs)
|    `---- connect4.py      # ConnectFour (7x7, drop coins)
```

10 Basic and Advanced Features

10.1 Basic Features

Two-Player Authentication

SHA-256 hashed passwords, registration and login flow, duplicate username prevention, and persistent storage in `users.tsv`. Implemented entirely in Bash.

Interactive Game Menu

Pygame-rendered menu with plain text game rows. Single click launches the selected game. Shows both player names in their respective colours.

Three Playable Games

All three games are rendered with Pygame GUI windows. No terminal-based gameplay. All boards are NumPy arrays. All win conditions use NumPy vectorized operations.

Game History Recording

After each game, `game.py` appends a row to `history.csv` containing Winner, Loser, Date, and Game name. The file is created with a header row on first use.

Terminal Leaderboard

After each game, the leaderboard is printed to the terminal sorted by the metric the players chose. Implemented in Bash with `awk` for per-game per-player statistics.

10.2 Advanced Features

Shared Base Class

`BoardGame` in `games/base_game.py` provides shared constants, layout helpers, and the `draw_top_bar()` method. All three game classes inherit from it, keeping game-specific code focused and minimal.

Fully Vectorized Win Detection

Tic-Tac-Toe and Connect Four win detection use zero Python loops for checking.

Drop Preview (Connect Four)

A ghost coin is rendered in the gap between the top bar and board, clamped to always be fully visible. The column hover highlight is clipped to the board area.

Valid Move Hints and Ghost Disc (Othello)

All legal placements are shown as dots. A semi-transparent ghost disc previews placement on hover. Flipped discs receive a gold highlight ring.

Post-Game Sort Picker

The post-game screen is two-phased: first the players pick the leaderboard sort metric (which triggers CSV write and leaderboard output), then they choose to play again or quit.

11 Hurdles Faced

- **Vectorizing win detection without sliding window view**: The standard approach for detecting runs in a board uses `np.lib.stride_tricks.sliding_window_view`, which was not permitted. The solution was the cumsum reset trick, which detects consecutive runs across entire rows/columns simultaneously using only `np.cumsum`, `np.where`, and `np.maximum.accumulate`.
- **Diagonal win detection without loops**: Diagonals have varying lengths and cannot be stacked directly.
- **Connect Four drop preview being hidden**: The ghost coin was being rendered before `draw_top_bar()`, which painted over it. Fixed by moving the preview draw

call to after the top bar, and clamping the coin's `y` coordinate to the gap between the bar and board.

- **Avoiding circular imports**: Game files need shared constants and the base class, but `game.py` dynamically imports game files. Placing `BoardGame` in `game.py` would create a circular import. The solution is `games/base_game.py`: a pure data and class file that imports nothing from the rest of the hub.
- **Othello turn-skip logic**: After placing, if the next player has no valid moves, the turn must skip back. If neither player can move, the game ends. This required careful ordering: switch turn → check valid moves → possibly switch back → check valid moves again → end game.

12 What I Would Do Differently With 10 More Hours

- **Fullscreen support**: Pygame's `pygame.display.toggle_fullscreen()` is trivial to add, but it only scales the window without scaling the board. A proper implementation would require making all layout constants dynamic (computed from live screen size at runtime) rather than module-level constants. With more time, this refactor would be worthwhile.
- **Matplotlib analytics**: The project specification requires a bar chart of top 5 players and a pie chart of most-played games. These are straightforward with `history.csv` and Matplotlib but were not implemented due to time constraints.
- **Animated coin drop in Connect Four**: Rather than the coin instantly appearing in its final row, a smooth fall animation along the column would improve the game feel.
- **Sound effects**: Short Pygame mixer sounds for placing marks, flipping discs, and winning would add polish.
- **Coin flip animation in Othello**: Smoothly transitioning disc colour when flipped (via a squash/stretch animation) would make the game more engaging.

references